TP1 – Seam Carving

Facundo Arechaga - Franco Bergman - Peter Ahn
Universidad Torcuato Di Tella

I. INTRODUCCIÓN Y MARCO TEORICO

Seam Carving (desarrollado por Avidan y Shamir) propone una técnica que consiste en el redimensionamiento consciente del contenido o también llamado Content-aware. Esta, permite eliminar filas o columnas enteras compuestas por pixeles conectados desde la parte superior de la imagen hasta la parte inferior, haciendo pasar este camino por zonas de "baja importancia visual". De esta manera, el método preserva las regiones más destacadas de la imagen, como, por ejemplo, caras de persona u otros objetos relevantes.

En comparación a métodos más tradicionales como el recorte o el escalado uniforme, que pueden eliminar información destacada o distorsionar la imagen, Seam Carving permite adaptar el tamaño de una imagen de una forma más inteligente. Esta técnica no solo es muy importante para reducir el tamaño de las imágenes o comprimirlas, sino también para el llamado "Image Retargeting", donde se busca mostrar imágenes sin distorsión en soportes de diversos tamaños (e.g. tablets, teléfonos móviles) manteniendo la calidad visual del contenido complementando así los mecanismos de diseño dinámico que estándares como HTML ya proveen para texto y maquetado, pero no para imágenes.

Este informe tiene como objetivo analizar distintos diseños de algoritmos que permiten resolver el problema de Seam Carving, incluyendo técnicas de fuerza bruta, backtracking con podas de optimalidad y programación dinámica. Las implementaciones se realizan tanto en C++ como en Python, lo cual permitirá no solo comparar el rendimiento de cada algoritmo sino también su eficiencia en la práctica.

El resto del informe se organiza de la siguiente manera: en la Sección 2 se describen los algoritmos propuestos, junto con su análisis de complejidad. En la Sección 3 se presentan detalles de implementación y decisiones de diseño adoptadas. En la Sección 4 se exponen los resultados experimentales y su análisis comparativo. Finalmente, en la Sección 5 se presentan las conclusiones del trabajo y posibles líneas de mejora futura.

II. DESCRIPCIÓN DEL PROBLEMA

El problema de Seam Carving consiste en encontrar una costura de mínima energía dentro de una imagen, con el objetivo de redimensionarla preservando las regiones de mayor importancia visual y descartando aquellas poco importantes.

Cada imagen se modela como una matriz de *n* filas y *m* columnas, donde cada posición (i, j) representa un píxel y tiene asociado un valor de energía $e_{ij} \geq 0$, que indica la importancia visual de ese pixel. Una costura vertical (seam) se define como una secuencia de píxeles:

$$s = ((1, c_1), (1, c_2), \dots, (1, c_n))$$

tal que para todo $i \in \{1, \dots, n-1\}$ se cumple:

$$|c_{i+1} - c_i| \leq 1$$

Es decir cada pixel de la costura debe ser adyacente vertical o diagonalmente al anterior. La energía total de la costura s se define como:

$$E(s) = \sum_{i=1}^{n} e_{i,c_i}$$

El objetivo del problema es encontrar una costura s tal que $E(s)$ sea mínima. La salida espera consiste en una secuencia de n enteros $c_1, c_2, \dots, c_n$ donde cada c_i indica la columna seleccionada en su respectiva fila i, representando en total, la costura de mínima energia.

6 7						
1,2	5,8	6,1	7,3	8,0	6,9	7,5
6,4	**1,1**	5,7	6,8	7,2	8,3	6,6
7,0	6,2	**1,3**	5,9	6,5	7,8	8,1
6,7	7,1	5,6	**1,0**	5,4	6,2	7,9
7,3	6,8	6,0	**1,2**	5,7	6,6	8,0
6,9	7,4	6,3	5,8	**1,1**	6,7	7,2

Considerando una imagen de tamaño 6 x 7, donde cada celda contiene un valor de energía (y donde cada fila contiene 7 valores de energía, donde, por ejemplo, la energía del píxel (3,4) es 5,6), una posible solución óptima es la costura definida por las columnas (1,2,3,4,4,5) cuya energía total es la suma de los valores correspondientes en cada fila: 1,2 + 1,1 + 1,3 + 1,0 + 1,2 + 1,1 = 6,9.

- Primera línea: El primer entero indica la cantidad de filas n (en este caso 6). El segundo entero representa la cantidad de columnas (en este caso 7).
- Las siguientes *n* líneas: Cada una contiene *m* valores reales no negativos $e_{i,1}. e_{i,2} \dots, e_{i,m}$, representando la energía de cada píxel en la fila *i*

III. MODELADO

Este problema podría pensarse <u>conceptualmente</u> como un grafo dirigido acíclico o DAG. Donde cada pixel (i, j) es un nodo distinto, y existen aristas dirigidas desde (i, j) hacia sus tres nodos adyacentes, es decir: $(i+1, j-1)$, $(i+1, j)$ e $(i+1, j+1)$, siempre y cuando estos se encuentren dentro de los limites de la matriz. Según este modelo, lo único que hay que hacer es encontrar la costura de mínima energía equivalente a encontrar el camino de mínimo costo desde cualquier nodo de la primera fila hasta cualquier nodo de la ultima fila. Sin embargo, dado que la estructura del grafo esta implícita en la propia matriz de energía, la implementación trabaja directamente usando la matriz sin construir el grafo explícitamente.

IV. ALGORITMO FUERZA BRUTA

El algoritmo de fuerza bruta resuelve el problema explorando exhaustivamente <u>todos</u> los caminos posibles desde cada columna de la primera fila hasta la última. Para cada pixel (i, j), se exploran recursivamente los vecinos validos de la siguiente fila y ahí se selecciona el camino de menor energía acumulada. Una vez explorados todos los caminos desde todas las columnas iniciales, se retorna la costura de menor costo global.

Mas precisamente, en encontrarSeamFuerzaBruta() se itera sobre cada columna j de la primera fila, tomándola como punto de inicio de una costura. Para cada columna j se llama a FuerzaBruta() que devuelve el costo y camino de la mejor costura que arranca en $(0, j)$. Si ese costo es menor que mejorCosto, se actualiza mejorCosto y mejorSeam. Por lo que, al finalizar el bucle, mejorSeam contiene efectivamente la costura mínima global.

En FuerzaBruta(), si se llegó a la última fila, se agrega la columna actual a res y se retorna la energía del píxel actual. En caso contrario, se exploran los vecinos validos de la fila siguiente, mientras este esté entre 0 y m – 1. Para cada vecino, se llama recursivamente a FuerzaBruta desde dicho vecino, obteniendo el costo y camino optimo. Si ese costo es mejor que mejorCosto, actualizamos tanto mejorCosto como res. Una vez explorados todos los vecinos, se coloca la columna actual al inicio de res y se retorna la energía del píxel actual más mejorCosto.

mejorCosto es local a cada llamada. A diferencia de BT donde mejorSuma es global. En FB cada llamada a FuerzaBruta desde una columna tiene su propio mejorCosto. Esto es consecuencia de no haber poda: en FB no hay ningún beneficio de compartir el mejor costo entre columnas porque nunca voy a poder descartar ramas.

Algorithm 1: EncontrarSeamFuerzaBruta(energía: matriz[n][m])

```
mejorCosto ← ∞;
mejorSeam ← [ ];
for j ← 0 to m − 1 do
    (costo, seam) ← FuerzaBruta(energía, 0, j);
    if costo < mejorCosto then
        mejorCosto ← costo;
        mejorSeam ← seam;
return mejorSeam;
```

Algorithm 1: FuerzaBruta(energía: matriz[n][m], fila: Z, col: Z)

```
if fila = n then
    return (energía[fila][col], [col]);
mejorCosto ← ∞;
res ← [ ];
for i ← −1 to 1 do
    nextCol ← col +i;
    if 0 ≤ nextCol < m then
        (costo, resAct) ← FuerzaBruta(energía, fila+1, nextCol);
        if costo < mejorCosto then
            mejorCosto ← costo;
            res ← resAct;
return (energía[fila][col] + mejorCosto, [col] + res);
```

i) <u>ANALISIS DE COMPLEJIDAD FUERZA BRUTA</u>

La función EncontrarSeamFuerzaBruta() llama a FuerzaBruta desde cada una de las *m* columnas que conforman la primera fila. Cada una de estas llamadas genera un árbol que se divide en 3 ramas: el adyacente izquierdo, el adyacente del centro y el adyacente derecho. Como se muestra en Figura 1 cada

vez que se baja una fila, multiplicas por 3. Por lo tanto, después de *n* filas hay 3^n caminos posibles. Y como esto se repite para cada columna de la fila 0 la complejidad resulta en:

$$T(n,m) = O(m \cdot 3^n)$$

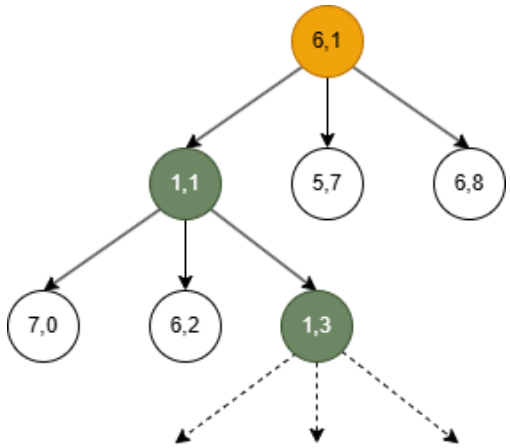


Figura 1. FuerzaBruta(energía, 1, j = 2). Cada nodo muestra el valor de energía

Es exponencial en *n* porque no reutiliza cálculos (es decir, si dos costuras pasan por el mismo pixel, lo recalcula desde cero nuevamente)

V. ALGORITMO BACKTRACKING

El algoritmo de backtracking resuelve el problema de forma muy similar a fuerza bruta. La diferencia es que se agrega una poda de optimalidad: si el costo acumulado hasta el pixel actual ya es mayor o igual al mejor costo global encontrado, se abandona esa rama sin explorar sus descendientes.

Esta poda se verifica antes de chequear si se llegó a la última fila. Es decir, si el costo acumulado ya supera el mejor conocido, se corta inmediatamente sin verificar la última fila. La alternativa era chequear primero el caso base y luego la poda, pero eso generaría trabajo innecesario en nodos que ya sabemos que no van a mejorar la solución.

Mas precisamente, en encontrarSeamBacktracking() se itera sobre cada columna c de la primera fila (de 0 a m – 1). Para cada c, se llama a encontrarSeamBackTracking_aux() que explora todos los caminos posibles desde $(0, c)$, actualizando mejor y mejorSuma cuando encuentra una costura de menor costo. Al terminar, mejor contiene la costura mínima global. Además, para la etapa de experimentación, en ese mismo punto se incrementa un contador de podas, con el objetivo de medir cuántas veces el algoritmo descarta ramas del árbol de búsqueda.

Luego, en encontrarSeamBackTracking_aux() se agrega la columna actual a actual y se suma la energía del pixel actual a sumaActual. Acá ocurre la poda: si sumaActual es mayor o igual a mejorSuma, se elimina la columna actual de actual y se retorna (esta rama no es una solución mejor). Si se llegó a la última fila y no se podó, entonces sumaActual es menor que mejorSuma. Se actualiza entonces mejorSuma y mejor. En caso contrario, se exploran los vecinos validos de la fila siguiente. Se llama desde

cada uno recursivamente a la función. Al finalizar, se elimina la columna actual de actual que restaura el estado del vector para que la siguiente rama lo encuentre limpio.

Algorithm 1: EncontrarSeamBacktracking(energía: matriz[n][m])

```
mejor ← [ ];
actual ← [ ];
mejorSuma ← ∞;
for c ← 0 to m − 1 do
    BackTracking aux(energía, 0, c, actual, mejor, mejorSuma, 0);
return mejor;
```

Algorithm 1: BackTracking_aux(energía: matriz[n][m], fila: *Z*, columna: *Z*, actual, mejor, mejorSuma, sumaActual)

```
agregar columna a actual;
sumaActual ← sumaActual + energía[fila][columna];
if sumaActual ≥ mejorSuma then
    elimnar último elemento de actual;
    return;
if fila = n − 1 then
    mejorSuma ← sumaActual;
    mejor ← actual;
else
    for d ← −1 to 1 do
        nextCol ← columna +d;
        if 0 ≤ nextCol < m then
            BackTracking aux(energía, fila+1, nextCol, actual, mejor,
              mejorSuma, sumaActual);
eliminar último elemento de actual
```

ii) ANALISIS DE COMPLEJIDAD BACKTRACKING

A diferencia del caso de fuerza bruta donde se exploran todos los caminos posibles (es decir, no se descarta nada y siempre se recorre todo el árbol), en backtracking se cortan las ramas que ya se sabe que no van a mejorar la solución. Sin embargo, puede ocurrir que la poda no se active nunca (e.g. Si las energías están ordenadas de menor a mayor hacia abajo) por lo que en el peor caso la complejidad será igual que en el caso de fuerza bruta:

$$T(n, m) = O(m \cdot 3^n)$$

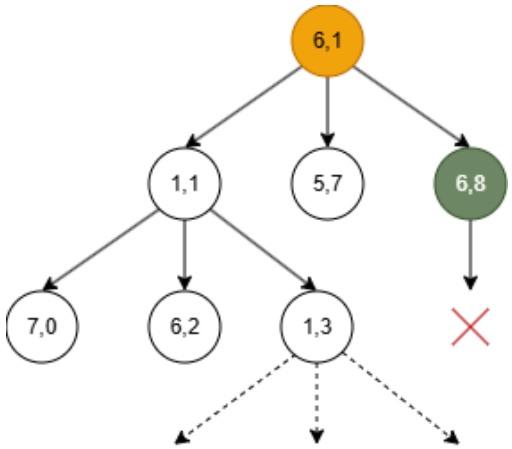


Figura 2. Backtracking. La X debajo de 6.8 muestra que esa rama se poda

Ahora bien, en el mejor caso la complejidad es:

$$T(n, m) = O(m \cdot n)$$

Si la primera costura ya es la óptima, todas las ramas siguientes se podan en la primera fila y solo se baja por un camino.

VI. ALGORITMO PROGRAMACIÓN DINAMICA

El algoritmo de PD evita el problema central de fuerza bruta y backtracking: el recalculo repetido de subproblemas. Si dos costuras distintas pasan por el mismo pixel (i, j), el subcamino hacia la ultima fila va a ser exactamente igual. Entonces, ese resultado solo necesita calcularse una vez y puede reutilizarse. El algoritmo construye un memoization M donde cada celda M[i][j] almacena la energía mínima acumulada de cualquier costura que pase por (i, j) hasta la ultima fila. Una vez llena la matriz, se reconstruye la costura optima desde la primera fila hasta la ultima, eligiendo en cada paso el vecino de menor valor en M.

En encontrarSeamPD se crea la matriz de memorización M de tamaño n*m. Las primeras n-1 filas se inicializan con -1 (indicando que todavía no se calcularon). La última fila se inicializa directo con los valores de energía de la última fila de la imagen ya que son el caso base (no tienen vecinos hacia abajo). Luego, se itera sobre cada columna c de la primera fila y se llama a encontrarSeamPD_aux() para llenar M. Gracias a la memoización, las celdas ya calculadas no se recomputan. Una vez llena M, se llama a reconstruccion_res para obtener la costura óptima.

En encontrarSeamPD_aux(): Si se llega a la última fila, se retorna ya que su valor ya está cargado en M. Si M[fila][columna] ¡= -1, la celda ya fue calculada en una llamada anterior, por lo que se retorna. Sino, se recorren los 3 vecinos de la fila siguiente. Para cada uno se hace recursivamente encontrarSeamPD_aux(), asegurando también que su valor en M este calculado. Una vez calculados todos los vecinos, se actualiza M[fila][columna] como la energía del píxel actual más la mínima energía de sus vecinos en M.

Por último, en reconstruccion_res(), se recorre la primera fila de M buscando la columna con el valor mínimo (se agrega ese índice a res ya que es el punto de inicio de la mejor costura). Luego se desciende fila por fila. En cada fila i se evalúan los 3 vecinos a la columna actual y se elige el de menor valor en M. Se actualiza la columna actual con el índice del vecino elegido y se agrega a res. Al terminar, res contiene los índices de columna de la costura optima.

El algoritmo de PD se dividió en tres funciones principales con responsabilidades separadas. encontrarSeamPD como función principal que inicializa M, encontrarSeamPD_aux que se encarga del llenado del memo y reconstrucción_res que reconstruye el camino optimo. Esta separación facilita la lectura, el testing individual y la identificación de errores durante el desarrollo.

Se utiliza un enfoque top-down (recursivo usando el memo) por su "similitud" con los algoritmos FB y BT, que también son recursivos. Esto facilita la comparación directa entre las tres implementaciones y hace más evidente sus relaciones.

iii) <u>ANALISIS DE COMPLEJIDAD PD</u>

El algoritmo utilizando programación dinámica evita el recalcula de subproblemas mediante una matriz de memoizacion M de tamaño $n \cdot m$ donde M[i][j] representa la energía mínima acumulada de cualquier costura.

En encontrarSeamPD_aux cada celda (i, j) de la matriz M se calcula exactamente una vez gracias al chequeo usando el memoization M[fila][columna] ¡= -1. Si ya fue calculada, se retorna sin hacer trabajo extra. Como hay $n \cdot m$ celdas y cada una trabaja en $O(1)$, el memo completo se llena en $O(n \cdot m)$. Es decir, aunque la función se llama desde las *m* columnas de la primera fila, la memoizacion garantiza que el trabajo no se multiplique por *m.*

La reconstrucción recorre la fila 0 para encontrar la columna de mínima energía en $O(m)$ y luego baja fila por fila eligiendo el vecino de menor valor en M entre 3 opciones, en $O(n)$. El costo total es $O(n + m)$, lo cual es despreciable frente al llenado de la matriz. Por lo tanto la complejidad temporal resulta en:

$$T(n, m) = O(n \cdot m)$$

La complejidad espacial es $O(n \cdot m)$ debido a la matriz M. A diferencia de FB y BT que solo necesitan $O(n)$ memoria para guardar el camino actual, PD almacena el resultado de cada pixel para no recalcularlo, pagando un costo en memoria a favor de una reducción exponencial de tiempo.

iv) COMPARACIÓN:

FB y BT comparten la misma complejidad en el peor caso ya que si la poda no se activa, backtracking recorre el mismo árbol. La diferencia entre ambos aparece dado que backtracking puede descartar grandes partes del árbol cuando encuentra soluciones buenas tempranamente, reduciendo significativamente el tiempo de ejecución.

Programación dinámica genera un gran salta respecto a los otros algoritmos. Evita el recalculo de subproblemas mediante la matriz M, reduciendo la complejidad de exponencial en *n* a polinomial. Para

dimensionar lo mucho más favorable que resulta este algoritmo: si tuviésemos una imagen de 20x20 pixeles, FB y BT en el peor caso deben explorar cerca de $20 \cdot 3^{20} \approx 700$ millones de caminos, mientras que en PD se resuelve con tan solo 400 operaciones. Esta brecha crece exponencialmente con n haciendo FB y BT terminen siendo inviables.

El costo espacial de PD es $O(n \cdot m)$ de memoria para la matriz M, a diferencia de BT y FB que solo requieren $O(n)$. Pero este costo es despreciable frente a la ganancia de tiempo de PD.

VII. DIFERENCIA DE IMPLEMENTACIONES

En líneas generales, las implementaciones tanto en C++ como en Python son muy similares. Aunque cambian algunos aspectos:

1) Especialmente cambia el manejo del paso por referencia (en Backtracking), con la variable mejorCosto. En C++ se pasa como double& mejorCosto, es decir, por referencia. Mientras que, en Python, esto no existe, por lo tanto, se simula el comportamiento usando una lista de un solo elemento tal que: mejor_costo = [inf].
2) En C++, el vector actual (en Backtracking) se modifica usando push_back y pop_back, restaurando el estado luego de cada búsqueda. En Python, en cambio, se construye el camino retornando [col] + mejor_sub_res, lo cual crea una nueva lista en cada llamada de la recursión.
3) En Programación Dinámica, hay una pequeña diferencia en cómo se construye M. En C++ hay que crear la matriz fila por fila con push_back. En Python, simplemente se usa [[-1.0] * m for _ in range(n-1)].
4) C++ compila código nativo y maneja directamente la memoria. Esto hace que sea mucho más rápido que Python. Esto también aplica para el caso de PD.
5) El resto de las diferencias son mayormente sintácticas y no tienen impacto.

Link conversación: https://claude.ai/share/9d32cc2f-64eb-4dfb-bde7-6c8ddefafdde. Se uso Claude para traducir los códigos de BT y PD a Python.

DIFICULTAD DE IMPLEMENTACION

Caso 1 – Fuerza Bruta: No resulta un algoritmo muy difícil de implementar ya que se basa en explorar todos los caminos posibles y quedarse con el mejor. No requiere estructuras o razonamientos extras como podas o subproblemas.

Caso 2 – Backtracking: Para nosotros, el más sencillo de implementar, ya que, utiliza un código muy similar al de Fuerza Bruta con el agregado de las podas. La poda de optimalidad se agrega con una sola condición al inicio de la función. La única dificultad estuvo en el manejo del vector actual para la reconstrucción del camino.

Caso 3 – Programación Dinámica: Con diferencia, el más complejo de implementar. Requiere razonar sobre la estructura de subproblemas, definir correctamente la recurrencia y diseñar la matriz de memoizacion. La implementación se divide en tres funciones con responsabilidades muy distintas. La parte más compleja es reconstruir el camino optimo: a diferencia de BT o FB donde el camino se construye naturalmente, en PD requiere una segunda pasada por la matriz M.

VIII. INSTRUCCIÓN DE COMPILACIÓN Y EJECUCIÓN

Requiere: **Compilador C++ con soporte para C++17, make, curl y Python.**

Desde la raíz del repositorio, ejecutar:

$$make$$

Esto descarga las librerías stb_image.h y stb_image_write.h en la carpeta *source/* y compila el ejecutable *seam.*

Ejecución – Modo numérico:

Recibe una matriz de energía en formato de texto y devuelve la costura óptima:

$$./seam\ --numerico\ <archivo>\ --algoritmo\ <fb|bt|pd>$$

Por ejemplo:

$$./seam\ --numerico\ input/ejemplo.txt\ --algoritmo\ pd$$

El resultado se imprime por pantalla y se guarda en: *output/numericos/seam_<algoritmo>.txt*

Ejecución – Modo imagen:

Recibe una imagen y elimina N costuras aplicando el algoritmo seleccionado:

$$./seam\ --imagen\ <archivo>\ --algoritmo\ <fb|bt|pd>\ --iteraciones\ <N>$$

Por ejemplo:

$$./seam\ --imagen\ img/foto.jpg\ --algoritmo\ pd\ --iteraciones\ 50$$

La imagen resultante se guarda en output/imagenes/resultado_<algoritmo>.png

Ejecución en Python:

Desde la carpeta source/ ejecutar:

$$python3\ main.py\ --numerico\ <archivo>\ --algoritmo\ <bt|pd>$$

IX. METODOLOGIA DE EXPERIMENTACIÓN

<u>Hipótesis:</u> La efectividad del backtracking depende directamente de la varianza de las energías de la imagen. Y cuando la varianza es baja, se espera que BT se comporte parecido a FB, mientras que en instancias con alta varianza logre podar una mayor cantidad de soluciones, mejorando su rendimiento.

<u>Hipótesis (Segunda parte):</u> PD presenta un mejor rendimiento que FB y BT, para matrices de cualquier varianza. Además, las implementaciones de los algoritmos ejecutados en Python resultan más lentas que las ejecutados en C++.

Para ello, se analizará cómo varía el desempeño del algoritmo de backtracking frente a fuerza bruta al modificar la dispersión de los valores de la matriz de energía. También veremos el comportamiento de programación dinámica.

La primera variable que tendremos en cuenta es como varían los valores de la matriz de energía de la matriz, es decir, usaremos tanto matrices con muchos valores muy parecidos como matrices con valores muy distintos.

Se generarán matrices de tamaño fijo $n = 12, m = 12$, ya que tamaños mayores vuelven inviable la ejecución de fuerza bruta y backtracking debido a su complejidad exponencial.

- Caso 1 (baja varianza): valores en el rango [1.0,1.1].
- Caso 2 (varianza moderada): valores en el rango [1.0,5.0].
- Caso 3 (alta varianza): valores en el rango [0.1,10].

3 3		
1,02	1,01	1,04
1,01	1,07	1,09
1,04	1,05	1,10

1. *Primer caso*

3 3		
1,32	4,34	2,38
1,99	3,09	1,09
2,04	3,05	4,10

2. Segundo caso

3 3		
0.23	9.10	5.05
2.33	3.98	8.97
1,04	0.99	4,10

3. Tercer caso

Para cada caso se generarán múltiples instancias independientes (5 por categoría), con el fin de evitar sesgos asociados a una única entrada.

Además, se analizara el resultado usando imágenes reales con distintos niveles de varianza. Y se analizan 5 ejecuciones usando PD y BT, para cada nivel de varianza, usando tanto Python como C++.

Se espera que:

- En el caso de baja varianza, todos los caminos deberían tener costos muy parecidos, lo que impide que backtracking encuentre rápidamente una solución significativamente mejor. Por lo tanto la poda será poco efectiva y el algoritmo tendrá un comportamiento cercano al de fuerza bruta.
- Con baja varianza, los costos de los distintos caminos tienden a ser cercanos entre sí, lo que reduce la capacidad de poda. En consecuencia, el algoritmo explora una mayor cantidad de soluciones.
- A medida que aumenta la varianza, es más probable encontrar caminos de bajo costo de forma temprana, lo que permite podar una mayor cantidad de soluciones. Esto debería resultar en una mejora significativa en el tiempo de ejecución del backtracking respecto a fuerza bruta.
- El algoritmo de programación dinámica debería ser el más eficiente en todos los casos, independientemente de la varianza.
- Las implementaciones en C++ presenten mejores tiempos de ejecución que las realizadas en Python, debido a las diferencias de eficiencia entre un lenguaje compilado y uno interpretado.

Metodología

- Hardware: Apple M2 Pro, 16 GB RAM, 512GB HDD.
- Sistema operativo: MacOS Ventura 13.4.
- Software: Python 3.12.2 y C++.

Datos utilizados:

- Las instancias se generan aleatoriamente utilizando distribuciones uniformes dentro de los rangos definidos para cada caso de varianza.
- Se realizaron experimentos adicionales sobre imágenes reales, las cuales se reescalan a matrices de energía de tamaño 12×12 para mantener la comparabilidad con las instancias sintéticas.
- Para el análisis comparativo entre lenguajes, se ejecutaron las mismas instancias tanto en C++ como en Python

Ejecución:

- Para cada instancia generada, los experimentos se ejecutan múltiples veces (5 repeticiones), registrando el tiempo de ejecución promedio.
- En el caso de la comparación entre lenguajes, se midieron los tiempos de ejecución de manera independiente en C++ y Python.

Reproducibilidad

- Para garantizar la reproducibilidad de los resultados: Se fija el tamaño de las instancias (n=12,m=12). Se especifican los rangos de generación de datos para cada caso. Se generan múltiples instancias por categoría. Se promedian múltiples ejecuciones bajo las mismas condiciones experimentales.

Análisis:

Se compararán los resultados obtenidos observando:

- La diferencia en tiempo de ejecución entre backtracking, fuerza bruta y programación dinámica.
- El impacto de la varianza en el rendimiento de los algoritmos usando tanto el modo numérico como el modo imagen.
- La consistencia de los resultados con la hipótesis planteada.
- Diferencia en tiempo de ejecución entre C++ y Python.

X. EXPERIMENTACIÓN Y ANALISIS DE RESULTADOS

Se presentan a continuación los resultados experimentales obtenidos mediante la evaluación de los algoritmos bajo distintos niveles de varianza en los valores de energía.

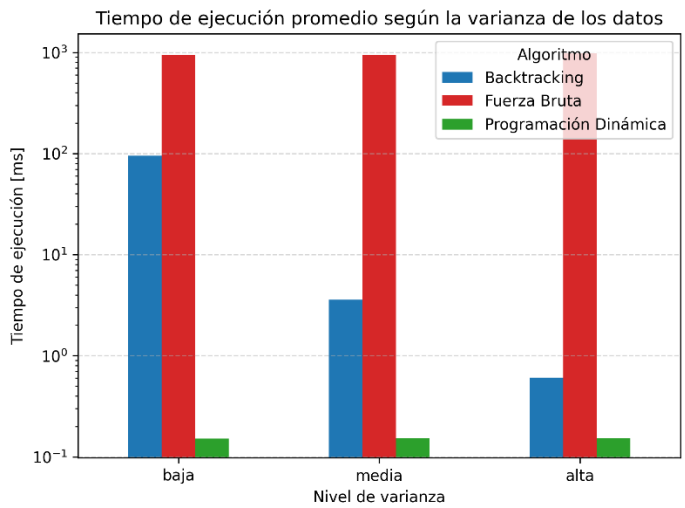


Figura 3. Tiempo de ejecución promedio BT vs FB vs PD

En la Figura 3. Se muestra el tiempo de ejecución promedio de todos los algoritmos para cada nivel de varianza. Se observa claramente que el algoritmo de FB mantiene un tiempo de ejecución constante en todos los casos, sin importar la varianza: esto es complemente esperado, ya que el algoritmo explora todas las soluciones posibles sin hacer podas.

Por otro lado, el algoritmo de BT presenta una gran mejora en su rendimiento a medida que aumenta la varianza de los datos. En particular, cuando la varianza es muy alta, el tiempo de ejecución disminuye muchísimo en comparación al caso de baja varianza.

Finalmente, PD resulta, con diferencia, el más eficiente en todos los escenarios analizados, lo cual es consistente con su complejidad polinómica y con su capacidad para evitar el recálculo de subproblemas.

En la Figura 4. se muestra la cantidad promedio de podas realizadas por el algoritmo BT para cada nivel de varianza. Contrario a lo que podíamos pensar inicialmente, la cantidad de podas es mayor en escenarios de baja varianza (nosotros pensábamos que tenía que ser al revés: es decir, a mayor cantidad de podas, mayor eficiencia). Esto ocurre, ya que, si los valores de energía son similares, el algoritmo explora una mayor cantidad de soluciones antes de hallar la óptima, lo que genera más oportunidades de aplicar la poda por optimalidad.

En cambio, cuando la varianza es más alta, los caminos tienden a ser mucho más distintos entre sí, lo que permite encontrar soluciones de bajo costo en etapas tempranas de la búsqueda. Por lo tanto, el algoritmo evita explorar gran parte del espacio de búsqueda, reduciendo la cantidad de soluciones evaluadas y la cantidad de podas necesarias

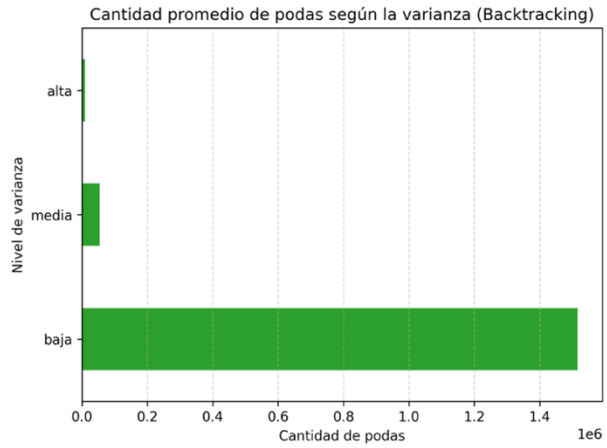


Figura 4. Cantidad de podas en BT según varianza

Con el objetivo de vincular los resultados experimentales con un caso más realista, se realizó una prueba sobre imágenes, comparando tres escenarios: uno con baja varianza en la matriz de energía, uno con varianza media y otro con alta varianza. Por cuestiones de viabilidad, cada

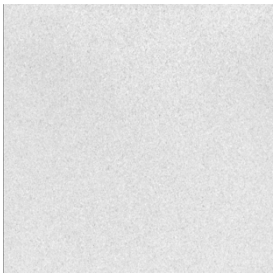
imagen se convierte a una matriz de energía 12x12 y se ejecuta una sola instancia de recorte.

	Baja varianza	Media varianza	Alta varianza
FB	962.79 ms	960.18 ms	965.81 ms
BT	3.16 ms	38.32 ms	3.03 ms
PD	1.93 ms	2.15 ms	2.17 ms

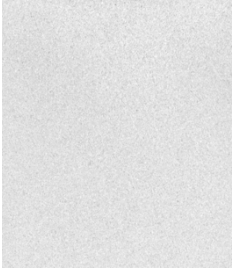
Caso de varianza media – ANTES

Caso de varianza media - DESPUES

Basado en los resultados presentados en la tabla, se observa algo distinto: BT mejora su rendimiento cuando la varianza aumenta de media a alta, sin embargo, en el caso de baja varianza se observan resultados similares al de varianza alta. Esto puede ser por varias razones: sugiere que el rendimiento del algoritmo no depende exclusivamente de la varianza de los datos, sino que puede verse influenciado por otros factores, como la estructura de la imagen, el orden de exploración de los caminos o que simplemente el experimente se vea condicionado por la transformación de imagen a matriz 12x12. Por otro lado, se mantiene prácticamente constante y PD presenta los mejores tiempos en todos los casos, como ya sabíamos.



Caso de varianza baja – ANTES



Caso de varianza baja- DESPUES

Los resultados obtenidos permiten validar parcialmente la hipótesis planteada. En una primera instancia, los resultados parecen confirmar que el rendimiento del algoritmo BT depende de la varianza de los valores de energía, con un rendimiento que mejora significativamente a medida que la varianza aumenta, lo cual es consistente con la idea de que mayor varianza mejora la identificación temprana de soluciones, y por lo tanto la poda. Sin embargo, al considerar cambios adicionales con imágenes reales,

hay situaciones en baja varianza que presentan tiempo de ejecución comparables con los de alta varianza.

Por otro lado, los resultados muestran que, incluso en escenarios de baja varianza, el algoritmo BT es mucho mejor que FB (Este comportamiento no era el esperado inicialmente, ya que se suponía que en el Caso 1: $FB \approx BT$).

Estos dos casos mencionados pueden sugerir que la eficiencia de BT depende no solo de la varianza, sino también de otros factores como el orden de exploración: es decir si se exploran los vecinos en orden (-1, 0, 1) o (1, 0, -1), por ejemplo.

Además, en escenarios de baja varianza se realizan más podas debido a que el algoritmo explora una mayor cantidad de soluciones antes de encontrar una óptima. En cambio, en escenarios de alta varianza, la identificación temprana de buenas soluciones reduce la necesidad de explorar el espacio de búsqueda, lo que disminuye tanto la cantidad de podas como el tiempo de ejecución.

Finalmente, el algoritmo de programación dinámica presenta el mejor rendimiento, manteniéndose prácticamente independiente de la varianza de los datos. Esto se debe a que su enfoque evita la exploración exhaustiva del espacio de soluciones mediante la reutilización de subproblemas, lo que lo convierte en una alternativa más eficiente frente a los enfoques exponenciales.

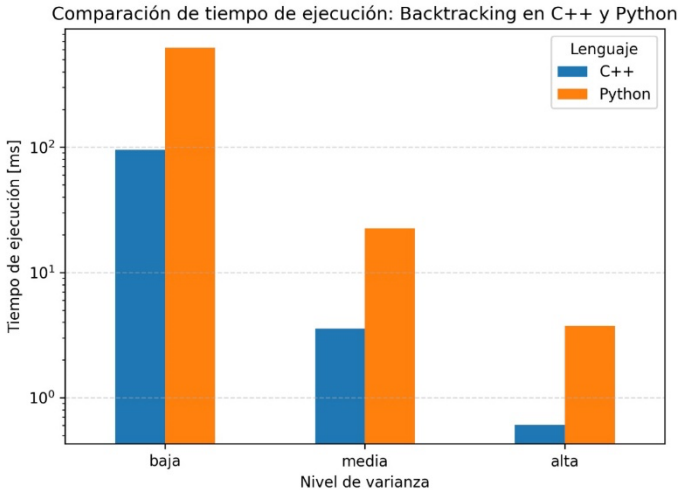


Figura 5. Tiempo de ejecución Python vs C++

En la Figura 5 se presenta una comparación del tiempo de ejecución entre las implementaciones en C++ y Python. Se observa en todos los casos, tiempos de ejecución menores en C++ comparado con Python.

Esta diferencia es consistente con las características de ambos lenguajes, ya que C++ es un lenguaje compilado, mientras que Python es interpretado, lo que introduce una sobrecarga adicional en la ejecución.

Sin embargo, es importante destacar que ambos lenguajes mantienen la misma tendencia en función de la varianza de los datos. De hecho, BT presenta una mejora en su rendimiento a medida que aumenta la varianza tanto en C++ como en Python, lo cual refuerza la validez de las conclusiones obtenidas previamente.

XI. CONCLUSION

En este trabajo se analizó el desempeño de los algoritmos de fuerza bruta, backtracking y programación dinámica en función de la varianza de los valores de energía. Los resultados permiten concluir que el algoritmo de backtracking presenta una mejora respecto a FB, especialmente cuando la varianza es alta. Se observa que el algoritmo de PD es, en todos los casos, el más eficiente, manteniendo tiempos de

ejecución significativamente menores e independientes de la varianza. La comparación entre implementaciones en distintos lenguajes muestra que C++ presenta tiempos de ejecución considerablemente menores que Python, aunque ambos mantienen las mismas tendencias de comportamiento en función de la varianza.

La hipótesis planteada se valida solo parcialmente. Se confirma que el rendimiento de BT depende de la varianza de los valores de energía (aunque no exclusivamente), mejorando cuando aumenta. No se verifica que en escenarios de baja varianza BT se comporte de manera similar a FB, ya que aun en estos casos presenta un mejor rendimiento. Tampoco se confirma que una mayor varianza implique una mayor cantidad de podas. Por el contrario, en instancias de alta varianza el algoritmo realiza menos podas, debido a que encuentra soluciones de bajo costo en etapas tempranas.

Además, a partir de los experimentos realizados sobre imágenes reales, se observaron casos en los que imágenes con baja varianza presentan tiempos de ejecución comparables a los de alta varianza. Este resultado indica que la varianza, si bien es un factor relevante, no resulta suficiente por sí sola para explicar completamente el comportamiento del algoritmo. En particular, aspectos como la estructura de los datos o el orden de exploración de los caminos pueden influir de manera significativa en la efectividad de la poda.

Estos resultados indican que el algoritmo de backtracking resulta una alternativa viable frente a fuerza bruta en la práctica, incluso en escenarios donde la varianza de los datos es baja. En particular, su capacidad para reducir el espacio de búsqueda efectivo lo convierte en una opción significativamente más eficiente para problemas de este tipo. Sin embargo, cuando se dispone de PD, este resulta claramente superior debido a su complejidad polinómica y a su capacidad de evitar la recomputación de subproblemas.

En base a los resultados obtenidos, se recomienda el uso de algoritmos como programación dinámica en aplicaciones reales, debido a su mejor desempeño y escalabilidad frente a enfoques exponenciales. También se recomienda el uso de C++ por sus mejores tiempos de ejecución.

En cuanto a las limitaciones del estudio, se destaca que los experimentos se realizaron sobre instancias de tamaño limitado (12×12), debido a la complejidad exponencial de los algoritmos analizados (en matrices 15x15 el algoritmo de FB tardaba varios minutos en completarse). Esto puede limitar la generalización de los resultados a instancias de mayor tamaño. Además, la generación de datos se basó en distribuciones uniformes simples, lo que podría no reflejar todos los escenarios posibles.

Como trabajo futuro, sería interesante extender el análisis a instancias de mayor tamaño utilizando algoritmos más eficientes como programación dinámica, así como explorar otros tipos de distribuciones de datos o estructuras específicas de matrices de energía. También podría analizarse el impacto de distintos órdenes de exploración en el desempeño del backtracking.